

THE \$3 BILLION WARNING

Governance Failures in Automated Finance and the Deterministic Infrastructure Response

Tanishq Dasari

Independent Researcher · AnimusLab

tan@animuslab.dev · [anchor](#) · animuslab.dev

\$3B+

Total Capital at Risk — 5 Cases

5

Regulatory Frameworks Satisfied

5

Months to Reconstruct Flash Crash

3.8+

Python Compatibility (anchor-audit)

Abstract

Five documented failures in automated financial systems have collectively destroyed or misallocated more than \$3 billion in institutional capital. In each case, the technology that caused the failure existed and was understood. The infrastructure to govern it in real time did not. This paper examines five case studies — Knight Capital Group (2012), the Flash Crash (2010), Zillow Offers (2021), TSB Bank (2018), and Citibank's Revlon transfer (2020) — and extracts a common structural pattern: these failures were not caused by systems doing unexpected things. They were caused by systems doing exactly what they were configured to do, in the absence of governance infrastructure capable of enforcing what they were **authorized** to do. We examine the current regulatory enforcement landscape — EU AI Act (enforcement begins August 2026), CFPB's 2024 nine-figure enforcement action against Goldman Sachs and Apple, and SEC 2026 examination priorities — and demonstrate that five major frameworks converge on one technical requirement: a tamper-evident, per-decision record of what an automated system decided, on what basis, under what authority, and why. We present Anchor, an open-source governance engine satisfying this requirement across all five frameworks simultaneously — a property we term **regulatory polyglottism**.

Keywords: AI governance, execution governance, financial services, regulatory compliance, EU AI Act, CFPB Regulation B, audit trails, deterministic enforcement, agentic AI

JEL Classification: G18, G20, K22, K24, O33

Availability: [pip install anchor-audit](#) · github.com/AnimusLab/anchor · anchorgovernance.tech

1. Introduction: The Pattern Behind the Losses

Between 2010 and 2021, five automated financial systems caused or contributed to losses and regulatory costs exceeding \$3 billion in aggregate. The systems involved were not experimental. They were production-grade infrastructure operated by sophisticated institutions — a market maker ranked among the largest in U.S. equities, a publicly traded real estate platform with billions in capital, a global systemically important bank, and a major UK retail bank undergoing a supervised IT migration.

In each case, the failure was investigated, documented, and attributed. In each case, the attribution pointed not to model inaccuracy or traditional software bugs, but to a common structural absence: the institution could not produce, in real time or after the fact, a complete and verifiable record of what its automated systems were authorized to do, what they actually did, and whether the two matched.

This paper argues that this absence — which we term the execution governance gap — is distinct from model governance (does the model produce accurate outputs), data governance (is data classified correctly), and content governance (is the output appropriate). Execution governance asks a more fundamental question: was this action authorized, and can we prove it?

The Core Distinction

"Is this output safe?" is a question about model quality. It is probabilistic and genuinely difficult. "Was this action authorized, and can we prove it?" is a question about governance and provenance. It is answerable today, with existing engineering techniques, without requiring any advance in the underlying models.

This question is answerable today with existing engineering techniques — cryptographic hashing, abstract syntax tree analysis, append-only audit logs, and structural output validation. The five cases examined below each illustrate a failure that execution governance infrastructure would have detected, blocked, or made immediately reconstructable.

1.1 The Distinction That Matters

Current AI governance tooling addresses important questions. Model monitoring platforms ask whether output quality is degrading. Bias classifiers ask whether decisions are discriminatory in aggregate. Content filters ask whether a specific output is appropriate. Each is valuable.

None of them asks what regulators are actually enforcing: for this specific decision, at this specific moment, was the action authorized under the institution's stated policy, and does a verifiable record of that authorization exist? This is the question Anchor is built to answer — a question about governance and provenance, not model quality.

2. Five Failures and the Governance Gap

The five cases below were selected because each is publicly documented at a level of detail supporting rigorous counterfactual analysis, and because each represents a distinct failure mode within the execution governance taxonomy.

Case	Year	Proximate Cause	Governance Gap	Total Cost
Knight Capital	2012	Deployment inconsistency + dormant code	No deprecated code detection; no fleet integrity verification	\$460M+ losses; \$12M SEC fine
Flash Crash	2010	Algorithmic feedback loop	No per-decision audit trail; 5-month reconstruction	Multi-participant systemic disruption
Zillow Offers	2021	Model drift under market change	No per-decision reasoning record; no exposure escalation	\$500M+ write-downs
TSB Bank	2018	Incomplete deployment verification	No configuration drift detection; no go/no-go attribution	£600M+ total impact
Citibank Revlon	2020	Authorization gap in payment system	No structured authorization record; no amount-range enforcement	\$500M permanently lost

2.1 Knight Capital Group — August 1, 2012

Classification: Deployment Integrity Failure · Deprecated Code Activation · Fleet Configuration Drift

On August 1, 2012, Knight Capital Group — at the time one of the largest market makers in U.S. equities, executing approximately 10-15% of daily trading volume in NYSE and Nasdaq-listed stocks — deployed new software to support the NYSE's Retail Liquidity Program across eight production trading servers. The deployment required updating code on all eight servers simultaneously. It reached seven of them.

The eighth server, through a single technician's oversight and the absence of any second-person verification requirement, continued running the prior version of the codebase. That prior version contained a dormant function called Power Peg — internal trading logic Knight had ceased using in 2003. Power Peg had been retained in the production codebase for nine years despite being operationally inactive. The new code inadvertently repurposed a configuration flag — the SMARS order router flag — for a new function. Power Peg, still present in the eighth server, still recognized that flag as its own activation signal.

When trading opened at 9:30 AM on August 1, the seven correctly-updated servers processed orders through the new Retail Liquidity Program normally. The eighth server activated Power Peg. Power Peg had, in 2003, included a cumulative quantity safeguard — a limit causing it to stop sending child orders once the parent order had been filled. In 2005, that safeguard was relocated within the codebase as part of a restructuring. It was never retested after the relocation. When Power Peg activated on August 1, 2012, the safeguard was no longer functioning.

For 45 minutes, Power Peg sent continuous streams of child orders — buying and selling across 154 stocks — without limit, without regard to whether parent orders had been filled, and without any position constraints. Knight's risk management systems generated continuous alerts, but trading operations staff were unable to identify which system was generating the orders or how to stop them. At approximately 10:15 AM, Knight's direct market access to the NYSE was terminated. By then, the firm had executed over 4 million trades, accumulating approximately \$3.5 billion in long positions and \$3.15 billion in short positions. Net loss: more than \$460 million — exceeding the firm's entire prior-year net income.

Knight Capital did not survive as an independent entity. It was acquired by Getco LLC in 2013 and the merged entity subsequently acquired by Virtu Financial. The SEC's administrative order (Release No. 70694, October 2013) found violations of Exchange Act Rule 15c3-5 and Regulation SHO, imposing a \$12 million civil penalty. The order specifically noted that a written protocol requiring retesting of Power Peg when it was last modified in 2005 could have identified that its position safeguard had been disabled — describing almost exactly what behavioral verification infrastructure provides.

Failure Component	What Was Missing	Governance Mechanism That Addresses It
Incomplete deployment to 8th server	No second-person deployment verification	Fleet integrity: hash-seal all servers before market open
Power Peg present in production	No deprecated code detection	Static blocklist: BLOCKER finding at CI for any deprecated module
Safeguard relocated but not retested	No behavioral re-verification post-modification	Diamond Cage: sandbox execution verifies behavior, not intent
45-minute exposure before detection	No real-time decision audit trail	Runtime DAC: every order generates a timestamped, chainable record

What Execution Governance Changes

Any one of three mechanisms interrupts this failure path before a single live order is placed: (1) Static constitutional analysis produces a BLOCKER finding at CI time for any deployment artifact containing a blocklisted deprecated module — by function name, flag-handling pattern, or structural signature. (2) Cryptographic fleet integrity verification prevents any server running code whose hash does not match the sealed current version from entering production — independent of whether Power Peg has been specifically identified as dangerous. (3) Behavioral verification confirms whether a code path behaves consistently with its specification under execution rather than inferring behavior from the fact that it once worked. The SEC order identifies all three missing controls by name.

2.2 The Flash Crash — May 6, 2010

Classification: Forensic Reconstruction Failure · Absence of Per-Decision Audit Trail

On May 6, 2010, U.S. equity markets experienced one of the most dramatic intraday collapses in their history. The Dow Jones Industrial Average fell approximately 9% within minutes — a decline of nearly 1,000 points — before recovering most of the loss within the same trading session. Individual stocks temporarily traded at prices ranging from one cent to \$100,000. The event erased and then restored nearly \$1 trillion in market value within a single afternoon.

The proximate trigger was a large institutional sell order — approximately \$4.1 billion in E-mini S&P 500 futures — executed through an automated algorithm that was calibrated to target a specific execution volume rather than price or time constraints. As this order hit a market already under stress from European sovereign debt concerns, it interacted with high-frequency trading systems to create a self-reinforcing feedback loop: rapid selling triggered withdrawal of HFT liquidity, which triggered further price declines, which triggered further automated selling.

A separate actor — Navinder Singh Sarao, a UK-based trader — was later found to have engaged in systematic spoofing activity in E-mini futures markets, placing and rapidly cancelling large orders to create false impressions of supply and demand that amplified the volatility. Sarao was not identified and charged until April 2015 — nearly five years after the event. The identification required years of retrospective investigation across multiple market venues, trading accounts, and jurisdictions.

The joint SEC-CFTC staff report — Findings Regarding the Market Events of May 6, 2010 — required five months to produce a complete account of an event that lasted approximately 36 minutes in its most severe phase. The investigation required reconstructing trading activity across multiple exchanges and thousands of individual securities from raw order and trade data. Everything investigators learned about why the market behaved as it did had to be inferred after the fact from the consequences of decisions — not from any record of the decisions themselves as the participants' systems understood them at the moment of execution.

The Core Governance Gap

No participant's algorithmic trading systems produced a structured, per-decision record at the time decisions were made. The audit trail that existed was the exchange's record of orders placed, modified, and cancelled — the outcomes of decisions — not the decisions themselves. For any future investigation of a specific institution's own systems, the difference between "we can answer that from our audit chain in seconds" and "we will need weeks to reconstruct" is a material reduction in regulatory and reputational exposure during the highest-scrutiny period an institution can face.

The claim here is narrow and important: Anchor's runtime Decision Audit Chain does not claim it would have shortened the market-wide five-month investigation, which spanned participants and venues most of which would not have been operating any such infrastructure. The claim is that for the portion of any future investigation concerning a specific institution's own systems, structured per-decision records transform forensic reconstruction from a multi-week manual process into a database query.

2.3 Zillow Offers — 2021

Zillow Offers was an algorithmic home-buying program in which pricing models generated cash offers on properties for Zillow to purchase and resell. During 2021, amid pandemic-driven housing market volatility, the historical pricing relationships the models relied on stopped holding in active markets. The algorithm continued generating purchase offers based on price relationships that no longer reflected current conditions.

By Q3 2021, Zillow disclosed it was holding large inventory purchased above resale value. The company took write-downs exceeding \$300 million in a single quarter, ultimately over \$500 million across the program's wind-down, shut down Zillow Offers entirely, and laid off approximately 25% of its workforce. The stock declined more than 50%.

This was, at its root, a model accuracy failure — and it is important to be precise that execution governance does not solve model accuracy. What was also absent was governance around the model's operation: no continuously queryable per-decision record of the model's stated reasoning for each purchase, and no constitutional requirement that purchases above a defined exposure threshold trigger structured human review.

What Execution Governance Changes

Structured Mode runtime governance would require each purchase decision to produce a machine-readable reason code and feature attribution at the time of decision. "On what basis is our model justifying offers in market X this week, compared to two weeks ago?" becomes a query against a structured log — not a question answerable only by aggregating financial outcomes months later.

2.4 TSB Bank IT Migration — April 2018**Classification: Configuration Governance Failure · Deployment Verification Gap · Information Accuracy at Decision Point**

In April 2018, TSB Bank undertook a migration of its core banking systems from infrastructure inherited from Lloyds Banking Group — its former parent — to a new platform developed by Banco Sabadell, its Spanish owner. The project had been in planning for years and represented a full separation of TSB's technology stack from Lloyds. The data migration itself completed successfully. The platform did not.

When the new system went live, it immediately failed across multiple dimensions. All of TSB's branches became unable to access customer accounts. A significant proportion of the bank's 5.2 million customers found themselves locked out of their own funds — unable to make payments, access balances, or conduct basic banking operations. The disruption lasted not hours or days but months. Some customers continued to experience problems until December 2018, eight months after launch. The prolonged outage triggered a wave of opportunistic fraud as criminals exploited customers who had lost visibility into their own accounts.

An independent review commissioned by TSB itself identified approximately 2,000 defects in the system at the time it went live. The board had been informed of approximately 800. The gap between 800 and 2,000 is not a reporting error — it is the difference between a board making a go/no-go decision with accurate information and a board making that decision without it. The review additionally found that only one of two data centers had been tested before launch. The decision to proceed without testing the second was justified internally on the grounds that if one center performed well, two centers would perform even better. The cross-configuration errors between the two centers, which caused the customer-facing failures, were therefore never identified because the test that would have found them was never conducted.

Internal Audit had raised concerns about the programme's readiness before launch, citing previous FCA enforcement actions related to poorly planned IT change management. The concern was noted. The risk was accepted. Neither the acceptance nor the basis on which it was made was documented in a form that made it individually attributable or independently verifiable after the fact.

The FCA and PRA jointly fined TSB £48.65 million in December 2022 — more than four years after the event — for operational risk management and governance failures. The FCA's executive director of enforcement described the failings as "widespread and serious" with "a real impact on the day-to-day lives of a significant proportion of TSB's customers, including those who were vulnerable." Combined with £32.7 million in customer redress and approximately £518 million in operational disruption and migration costs, total financial impact exceeded £600 million.

What Execution Governance Changes

Three specific absences drove this failure. First, no mechanism confirmed that the system being deployed matched the system that had been reviewed and approved across all relevant behavioral properties — specifically the dual-datacenter configuration. Static constitutional analysis would surface this configuration drift as a BLOCKER before go-live. Second, the board was asked to make a go/no-go decision and received materially incomplete information. A structured readiness record requirement — listing all known defects by severity, tested versus untested components, and all open risks with attribution — would have surfaced the 2,000/800 discrepancy at the decision point. Third, the governance floor invariant ensures that any decision to proceed despite open findings is individually attributed, signed, and logged as a sealed event — making it accountable from the moment it is made rather than four years later under regulatory investigation.

2.5 Citibank Revlon Transfer — August 11, 2020

Classification: Authorization Gap · Structured Output Absence · Amount-Range Enforcement Failure

On August 11, 2020, Citibank, acting as administrative agent for a \$894 million syndicated loan to Revlon Inc., set out to make a routine \$7.8 million interest payment to Revlon's lenders. A Citi employee, manually configuring the payment in the bank's loan processing software, selected an option that caused the system to treat the full principal balance as the payment amount. Three colleagues — acting as the institution's authorization safeguards for the transaction — reviewed and confirmed the payment before transmission. None of the three identified the error.

Why did three experienced financial professionals confirm a \$893 million payment they believed was a \$7.8 million payment? Because the authorization system presented them with a payment amount, not a structured record of what they were authorizing. What was required for meaningful authorization — payment type (interest versus principal), amount relative to the authorized range for this payment cycle, scheduled maturity of the underlying obligation, source of funds, and explicit confirmation that the amount was within policy bounds — was not structurally required by the system before confirmation could proceed.

The \$893 million transmission was, furthermore, the exact amount to the penny that the lenders were owed on the loan principal — a coincidence that led several lenders to believe the payment was an intentional early prepayment by Revlon. When Citibank identified the error and requested recovery, approximately \$390 million was returned by lenders who recognized the mistake. Ten lenders holding approximately \$500 million refused, arguing they were legitimately owed the funds. Their legal position was not unreasonable — they had received the exact amount they were owed, appearing to arrive as a legitimate prepayment.

In February 2021, Judge Jesse Furman of the U.S. District Court for the Southern District of New York ruled in the lenders' favor under the discharge-for-value doctrine: lenders who received a payment they were legitimately owed, without knowledge that it was transmitted in error, were not required to return it. Citigroup CEO Jane Fraser described the incident as "a massive, unforced error" and "one of the biggest blunders in banking history." After two years of litigation, Citibank settled with all ten lenders in December 2022. The majority of the mistaken principal payments were ultimately recovered. Approximately \$500 million was not.

What Execution Governance Changes

This failure is a direct application of the No-Prose Rule at the payment authorization layer. Structured Mode applied to any payment instruction above a defined threshold would require a machine-readable authorization record specifying: payment type (interest versus principal), payment amount relative to the authorized range for this payment cycle, and explicit confirmation that the amount is within policy bounds. A payment of \$893 million against an authorized cycle payment of \$7.8 million — a ratio of 114:1 — produces a BLOCKER finding before transmission. The three reviewers who confirmed the Citibank payment were not inattentive. They were working with a system that did not give them the information necessary to catch the error.

3. The Regulatory Enforcement Landscape

The five failures above produced regulatory responses across four jurisdictions. Those responses, read together, describe a convergent technical requirement. This section demonstrates that five major frameworks — independently drafted, addressing different jurisdictions and different AI use cases — are each, in their own regulatory language, demanding the same underlying infrastructure artifact.

3.1 EU AI Act (Regulation 2024/1689)

The EU AI Act classifies credit scoring, anti-money-laundering monitoring, and fraud detection as high-risk AI systems under Annex III. Article 12 requires that high-risk systems enable automatic recording of events throughout the system's lifetime — logs that are automatically generated, enable tracing of events throughout the period of use, and are tamper-evident. Article 13 requires sufficient transparency for affected persons to exercise their rights. Article 9 requires a risk management system documented throughout the system's lifecycle.

Enforcement on high-risk systems begins August 2026. Penalties reach €30 million or 6% of global annual revenue, whichever is higher. The specific technical requirement Article 12 creates — automatically generated, tamper-evident logs — is not satisfied by a log file. It requires a log structurally protected against alteration, with a mechanism for demonstrating that protection to an auditor. This is a cryptographic requirement stated in regulatory language.

3.2 CFPB Regulation B and the Goldman Sachs Precedent

The CFPB's October 2024 enforcement action against Goldman Sachs and Apple — totaling over \$89 million — established a critical precedent. The violation was not that the underlying credit model was biased in aggregate. The finding was that AI-assisted credit decisions could not be explained at the individual-decision level on demand.

Regulation B requires that applicants denied credit receive a statement of specific reasons for the denial. CFPB Circular 2024-03 explicitly extends these requirements to AI-assisted lending, and states that a creditor cannot satisfy adverse action notice requirements by providing reasons that are too vague or that do not accurately reflect the factors that actually drove the decision. A credit decision system returning prose explanation rather than machine-readable reason codes is, by construction, non-compliant — regardless of how accurate or unbiased the model is.

3.3 SEC 2026 Examination Priorities

The SEC's Division of Examinations named AI governance a top priority for 2026 examinations of registered investment advisers and broker-dealers. Stated priorities include: accuracy of marketing claims about AI capabilities relative to actual capabilities of deployed systems (the "AI washing" concern); audit trails of AI systems used in trading and investment decisions; and governance structures surrounding AI systems. The audit trail priority is the directly relevant requirement — the SEC's concern is whether an institution can demonstrate, from verifiable records, what its AI systems actually did and under what governance they operated at the time.

3.4 Reserve Bank of India FREE-AI Report (August 2025)

The RBI's Framework for Responsible and Ethical Enablement of AI issued 26 recommendations for AI in Indian financial services. Recommendation 7 requires per-decision audit trails reportable to the central bank's CIMS portal in a specified format. Recommendation 14 requires explainability at the level of individual decisions with machine-readable reason codes. Recommendation 19 requires mandatory algorithmic fairness audits for credit and lending systems. The CIMS portal has a defined submission schema — making "explainability" a structured data requirement, not a conceptual one.

3.5 NIST AI Risk Management Framework

The NIST AI RMF establishes Govern, Map, Measure, and Manage as the four core functions of AI risk management. The Govern function requires documented AI policies and accountability structures. The Manage function requires continuous monitoring of AI system behavior and documented response procedures. Together, these describe an ongoing operational requirement — not a one-time assessment — for institutions deploying AI in high-stakes contexts.

3.6 The Convergence

Read in isolation, these are five separate compliance obligations from five separate bodies across four jurisdictions. Read together, they describe a single underlying requirement in five regulatory dialects:

An institution must be able to produce, on demand, a record of what an AI system decided, on what basis, under what authority, and why — and that record must be tamper-evident.

4. Anchor: The Deterministic Infrastructure Response

4.1 Architecture Overview

Anchor is an open-source governance engine (pip install anchor-audit, Apache 2.0 license, Python 3.8+) enforcing a single, cryptographically signed rule set — a constitution — across two layers.

Layer 1 (Static) operates before deployment. It parses AI-adjacent source code into abstract syntax trees using Tree-sitter and evaluates those trees against the constitution's rules. This runs

in continuous integration, as a pre-commit hook, or on demand, producing a findings report before code reaches production.

Layer 2 (Runtime) operates on live AI inference. A lightweight decorator intercepts model calls, hashes the input and output, evaluates the output against the constitution, and writes the result to an append-only, cryptographically chained audit log — the Decision Audit Chain (DAC).

The architectural significance of one constitution, two layers: a rule defined once — for example, "credit decisions must include a machine-readable reason code, not prose" — is enforced both as a static check on the code that generates credit decisions and as a runtime check on the actual decisions produced. There is no drift between the static policy and the runtime policy, because they are the same policy.

4.2 The Decision Audit Chain

Each intercepted runtime call produces an `AuditEntry`: timestamp, model provider and version, active jurisdiction, git commit of the calling code, compliance status, SHA-256 hashes of the input and output, and specific rules evaluated. These entries form the Decision Audit Chain — an append-only log in which each entry's hash incorporates the hash of the preceding entry, such that altering any historical entry invalidates every entry that follows. Each entry additionally carries an HMAC-SHA256 signature.

The combination of hash-chaining and per-entry signing provides evidence binding, tamper detection, and non-equivocation — applied to individual AI decisions in a financial context, written to local disk by default with no external data transmission required.

4.3 Regulatory Polyglottism

The `AuditEntry` exposes jurisdiction-specific output methods translating one underlying record into the format a given regulator requires:

```
entry.adverse_action_reasons() # CFPB Regulation B reason codes
entry.cims_payload()           # RBI CIMS-format report
entry.eu_article12_record()    # EU AI Act Article 12 log entry
```

A single underlying decision — a loan denial — can be rendered simultaneously as a CFPB-compliant adverse action notice, an RBI CIMS submission, and an EU AI Act Article 12 log entry. This is not translated as a separate step. It is because the frameworks, read carefully, are independently demanding the same underlying artifact in different regulatory language. The polymorphic output methods are presentation layers over one underlying record.

We term this property regulatory polyglottism: one governance infrastructure layer, one underlying record, simultaneous satisfaction of multiple jurisdictional requirements.

4.4 The No-Prose Rule and the Goldman Sachs Connection

The CFPB's enforcement action against Goldman Sachs was, at its structural core, a violation of what Anchor terms the No-Prose Rule: AI-assisted credit decisions returned explanations in unstructured natural language rather than machine-readable reason codes and feature attribution. The violation was the absence of required structure, not the content of the decisions.

Anchor's Structured Mode enforces this structurally. In Structured Mode, a decision is not valid unless it includes a machine-readable reason code and feature attribution. Returning unstructured prose for a high-stakes decision is itself a BLOCKER finding by construction — the absence of the required structure is detected with certainty because it is a question about the shape of the output, not its content.

4.5 GOVERNANCE.lock and the Governance Floor Invariant

A governance system that can be silently weakened by the people it governs is documentation, not governance. GOVERNANCE.lock seals the constitutional files via SHA-256 hash, verified on every scan. Any modification — including a formatting change — produces a hash mismatch that aborts the scan with an integrity violation report. An institution can demonstrate, cryptographically, that the governance rules in effect at the time of a given decision are the rules that were authored and sealed.

The governance floor invariant ensures that an institution's local policy overrides can raise the severity of a rule above the constitutional floor, but the engine rejects any attempt to lower a rule's severity below the floor set by the signed constitution. This is enforced at the engine level, not as a policy convention.

4.6 Performance

A governance layer that adds material latency to every model call is not viable at institutional scale. Anchor's core operations were benchmarked on Python 3.14.4, Intel Core 13th/14th Gen (Raptor Lake), Windows 11, 16GB RAM, with 10,000 iterations per suite following 200-iteration warmup:

Operation	Mean Latency	P95	P99	Throughput
SHA-256 hash generation	0.73 μ s	0.80 μ s	0.90 μ s	1,360,544 / sec
HMAC-SHA256 chain verification	3.71 μ s	3.90 μ s	4.00 μ s	269,684 / sec
Full policy evaluation (118 rules)	4.52 ms	5.24 ms	7.00 ms	221 / sec
End-to-end receipt generation	12.38 ms	18.71 ms	21.26 ms	81 / sec

The cryptographic primitives add approximately four microseconds to a decision pipeline — negligible relative to any model call latency of tens to hundreds of milliseconds. The dominant cost is policy evaluation at 4.5ms, which in production runs at check-time against code changes rather than on the per-inference hot path. anchor-audit requires Python 3.8 or higher, covering virtually all production Python environments currently deployed in financial institutions.

5. Regulatory Compliance Mapping

The following tables map Anchor's mechanisms to specific provisions of the five frameworks. The mapping was produced by reading the cited regulatory text and identifying the specific AuditEntry field or constitutional mechanism that satisfies each provision.

Important limitation: this mapping has not been reviewed by external legal counsel or validated by any regulatory body. What Anchor provides is the technical substrate — the logs, the structure, the cryptographic guarantees — that makes a conformity assessment possible to perform. Whether a given regulator's examiners would accept a given field as sufficient evidence of compliance with a given provision is a question this table cannot answer.

Framework	Provision	Requirement	Anchor Mechanism
EU AI Act	Art. 12	Automatic, tamper-evident logging	Decision Audit Chain; hash-chaining + HMAC
EU AI Act	Art. 13	Transparency / explainability	No-Prose Rule; structured reason codes
EU AI Act	Art. 9	Risk management system	Signed constitution.anchor; GOVERNANCE.lock
CFPB Reg. B	Adverse action	Reason specificity for AI credit decisions	adverse_action_reasons(); ReasonCode field
CFPB Reg. B	Explanation	No generic denial reasons	FeatureAttribution; structural enforcement
RBI FREE-AI	Rec. 7	Per-decision audit trail, CIMS-reportable	DAC; cims_payload()
RBI FREE-AI	Rec. 14	Explainable credit decisions	No-Prose Rule + FeatureAttribution
RBI FREE-AI	Rec. 19	Algorithmic fairness audits	ETH-001 prohibited-proxy taxonomy
SEC 2026	Audit trails	AI trading system audit trails	Full DAC; git_commit provenance
SEC 2026	AI washing	Marketing claims match actual controls	GOVERNANCE.lock sealed rule set as evidence
NIST AI RMF	Govern 1.1	Documented governance policies	Signed constitution.anchor
NIST AI RMF	Manage 4.1	Continuous monitoring	AnchorRuntime continuous evaluation

5.1 Field-Level Mapping

The following table maps individual AuditEntry fields to the specific regulatory requirements they satisfy, enabling compliance reviewers to verify the mapping mechanism-by-mechanism.

AuditEntry Field	Contents	Regulatory Requirement Satisfied
entry_id	UUID, unique per decision	EU AI Act Art. 12 unique log record; RBI CIMS identifier
timestamp	ISO 8601 UTC	EU AI Act Art. 12 automatic timestamp; FCA oversight-record timing
provider	Model/provider + version	FCA model traceability; SEC audit trail
git_commit	Code provenance of calling application	FCA model traceability; NIST AI RMF Measure 2.5
status, is_compliant	Constitutional evaluation outcome	NIST AI RMF Manage 4.1 continuous monitoring
ReasonCode	Machine-readable decision reason	CFPB Regulation B adverse action; RBI FREE-AI Rec. 14
FeatureAttribution	Specific input factors driving decision	CFPB Reg. B reason specificity; EU AI Act Art. 13
violations (ETH-001)	Prohibited-proxy terms detected	RBI FREE-AI Rec. 19; ECOA proxy-variable concerns
chain_hash	Hash-chain linkage per entry	EU AI Act Art. 12 tamper-evident logging
signature	HMAC-SHA256 per-entry signature	Non-repudiation; SEC audit-trail integrity
GOVERNANCE.lock	SHA-256 sealed rule set	NIST AI RMF Govern 1.1; SEC AI washing defense

6. Implementation Pathway

6.1 Integration Effort

For static (Layer 1) governance, integration into an existing Python codebase (3.8+) requires three commands:

```

pip install anchor-audit
anchor init --all          # installs all domains and regulator files (~2 minutes)
anchor check .             # runs the first scan immediately

```

For runtime (Layer 2) governance, a single import activates interception for calls made through registered SDK integrations (OpenAI, Anthropic, LangChain-based pipelines):

```

import anchor.runtime      # auto-activates; patches registered AI SDKs

```

For high-stakes decision functions requiring Structured Mode — credit, lending, trading — the relevant functions are wrapped with the @anchor.enforce decorator:

```
@anchor.enforce(mode="structured", jurisdiction="CFPB")
def evaluate_credit_application(applicant_data):
    return llm.call(prompt_template, applicant_data)
```

6.2 Deployment Models

For institutions with data residency requirements under EU GDPR, India's DPDP Act, or internal policies prohibiting transmission of source code or customer data externally, Anchor supports air-gapped initialization:

```
anchor init --no-sign # bundles all governance files locally at install time
```

The Decision Audit Chain writes exclusively to local disk (.anchor/runtime_chain.jsonl) regardless of configuration. No prompt, response, or audit data is transmitted over a network in the default audit-write path. The Sovereign Relay mechanism allows an authorized party — a regulator, internal audit — to verify governance is operating correctly without receiving the underlying data, via a chain hash transmitted externally rather than the full record.

6.3 Governance Assessment Structure

A governance assessment — running Anchor against an institution's existing AI-adjacent codebase, mapping findings to the relevant regulatory framework, and producing a findings brief — can be completed in three to five business days with read-only access and no changes to production systems.

At the end of a full integration, an institution has: a CI/CD gate blocking deployments containing governance violations; a running hash-chained audit log of AI decisions in scope; a governance_audit.md report suitable for an internal audit function or external examiner; and a GOVERNANCE.lock-verified statement of which governance rules were in effect — independently checkable rather than dependent on internal documentation.

7. Scope of Deterministic Guarantees

Throughout this paper we use the word "deterministic" and we are precise about its scope. The claim is: for the class of violations represented in an organization's sealed constitution, and detectable by the corresponding mechanism in mitigation.anchor, Anchor's enforcement is deterministic — the rule fires if and only if the pattern is present, and (for BLOCKER-severity findings in CI) the commit is blocked if and only if the rule fires.

This guarantee has explicit boundaries. A developer can annotate a finding as an authorized exception — this is logged and individually attributed, but does not block the commit. A violation pattern not yet encoded in the mitigation catalog will not be detected by the corresponding static rule. A pattern existing only as a consequence of runtime control flow not visible in the AST falls to Layer 2's runtime interception.

The Correct Summary

Anchor deterministically enforces the violations its constitution defines. It does not claim to anticipate violations its constitution does not yet define. The gap between these two sets is closed incrementally, the same way a security team maintains a vulnerability database — explicitly, auditably, and with each addition individually attributed.

8. Comparison to Existing Approaches

A natural reaction to Anchor is to place it in the same category as existing AI governance tooling and ask how it compares feature-for-feature. The more useful question is which question a given tool is designed to answer — because several well-established categories answer fundamentally different questions than the one this paper has argued is currently unaddressed.

The five systems below represent major, widely-deployed categories of AI governance tooling in enterprise environments — not Anchor's closest competitors by feature count, but the categories an institution evaluating this space will encounter:

Tool / Category	Core Question It Answers	What It Governs	Answers the Authorization Question?
Microsoft Purview	What happened to this data, and who has access?	Data classification, lineage, compliance reporting	No — governs data, not AI decisions
Fiddler AI / Model Monitors	Is this model's output quality degrading?	Model drift, explainability dashboards, aggregate fairness	No — governs model quality, not authority
Weights & Biases / MLOps	What happened during training and evaluation?	Experiment tracking, model lifecycle, training provenance	No — governs training, not execution
Guardrails AI / NeMo	Does this specific output conform to a schema?	Prompt and output validation at generation time	Partially — schema only, not authorization chain
Llama Guard / Content Classifiers	Is this content safe to surface to a user?	Content moderation, safety classification	No — governs content, not authority
Open Policy Agent (OPA)	Does this request comply with defined policies?	Infrastructure policy enforcement, API authorization	Partially — policy enforcement, not AI-specific audit chain
Anchor	Was this action authorized, and can we prove it?	Execution authority, decision provenance, cryptographic audit	Yes — this is the primary design goal

Each of the established categories above asks a question that precedes or accompanies a model's output. Purview asks about the data the model touches. Fiddler asks about the model's behavior over time. Weights & Biases asks about how the model came to exist. Guardrails and Llama Guard ask about the specific output the model just produced.

Anchor asks a question that comes after all of these have been satisfied: given that an output exists, and given that it may even be perfectly accurate, well-formed, and appropriate in content

— is the system permitted to act on it, and can that permission be proven? This is orthogonal to model quality. A system with arbitrarily good model quality can still take an action it was never authorized to take. A system with mediocre model quality can be governed perfectly well at the execution layer while its quality is improved separately.

This is why Anchor does not displace the tools above in a typical deployment. An institution operating agentic AI at scale will reasonably run data governance tooling, a model-monitoring platform, an MLOps platform, and a content classifier — alongside Anchor for execution governance. These are complementary layers of a stack, not competing implementations of the same layer.

9. Conclusion

The five automated financial system failures examined in this paper — costing more than \$3 billion in aggregate — share a common structural pattern: the absence of infrastructure capable of enforcing, in real time and with a verifiable record, the boundary between what an automated system was authorized to do and what it actually did. Five regulatory frameworks have independently converged on the same response — an institution must produce, on demand, a tamper-evident per-decision record of what its AI systems decided, on what basis, under what authority, and why.

This requirement is satisfiable today. Not with a better classifier, not with more sophisticated safety training, but with well-understood software engineering and cryptographic techniques applied to the specific layer where the five failures above occurred: the layer between "the system produced an output" and "the system was authorized to act on it."

The EU AI Act's enforcement date is within the current planning horizon of every institution operating in the European market. The CFPB has already demonstrated enforcement with nine-figure penalties. The SEC has named AI governance an examination priority for the year now underway.

The institutions that close the execution governance gap before an incident forces the question will be in a materially different position — both operationally and with their regulators — than those that close it afterward.

Aug 2026

EU AI Act Enforcement Begins

€30M / 6%

Maximum EU AI Act Penalty

\$89M+

CFPB Precedent Enforcement Fine

pip install anchor-audit

Anchor Install Command

Anchor is available as open-source software under the Apache 2.0 license at github.com/AnimusLab/anchor and via `pip install anchor-audit`. Interactive policy evaluation is available at anchorgovernance.tech.

References

- [1] European Parliament and Council. Regulation (EU) 2024/1689 (Artificial Intelligence Act). Official Journal of the European Union, 2024.
- [2] Reserve Bank of India. Framework for Responsible and Ethical Enablement of AI (FREE-AI). Technical report, RBI, August 2025.
- [3] Consumer Financial Protection Bureau. CFPB orders Apple and Goldman Sachs to pay over \$89 million. Enforcement press release, October 2024.
- [4] Consumer Financial Protection Bureau. Circular 2024-03: Adverse action notification requirements and AI. CFPB Circular, 2024.
- [5] U.S. Securities and Exchange Commission. 2026 Examination Priorities. SEC Division of Examinations, January 2026.
- [6] National Institute of Standards and Technology. AI Risk Management Framework (AI RMF 1.0). NIST AI 100-1, January 2023.
- [7] U.S. Securities and Exchange Commission. In the Matter of Knight Capital Americas LLC. Release No. 70694, File No. 3-15570, October 16, 2013.
- [8] U.S. Securities and Exchange Commission and U.S. Commodity Futures Trading Commission. Findings Regarding the Market Events of May 6, 2010. Joint Report, September 30, 2010.
- [9] U.S. Department of Justice. U.K. Trader Charged with Illegally Manipulating Stock Market. Press release, April 2015.
- [10] Zillow Group, Inc. Form 10-Q for the Quarterly Period Ended September 30, 2021. SEC EDGAR, November 2, 2021.
- [11] Financial Conduct Authority and Prudential Regulation Authority. TSB fined £48.65m for operational resilience failings. FCA Press Release, December 20, 2022.
- [12] TSB Bank. Independent Review of TSB's IT Migration Programme. Slaughter and May, 2019.
- [13] Citibank N.A. v. Brigade Capital Management, L.P. U.S. District Court, SDNY, No. 20-06539, February 16, 2021.
- [14] Financial Conduct Authority. Artificial intelligence: How FCA principles apply. Discussion Paper DP 5/4, 2024.
- [15] L. Kao. Constant-size cryptographic evidence structures for regulated AI. arXiv:2511.17118, 2025.
- [16] B. Schneier and J. Kelsey. Secure audit logs to support computer forensics. ACM TISSEC, 2(2):159–176, 1999.
- [17] S. Crosby and D. Wallach. Efficient data structures for tamper-evident logging. USENIX Security, 2009.
- [18] M. Brunsfeld et al. Tree-sitter. tree-sitter.github.io, 2024.
- [19] FINOS Foundation. Architecture as Code. github.com/finos/architecture-as-code, 2024.
- [20] I. D. Raji et al. Closing the AI accountability gap. ACM FAccT, 2020.

About the Author

Tanishq Dasari is an independent researcher and founder of AnimusLab, an open-source AI governance infrastructure organization. AnimusLab is an active contributor to FINOS Labs and contributed to the DTCC Hackathon 2026, with a merged contribution implementing automated mapping from architectural components to governance mitigations within the FINOS reference architecture. Correspondence: tan@anchorgovernance.tech. Anchor engine: github.com/AnimusLab/anchor. Research: animuslab.dev.